

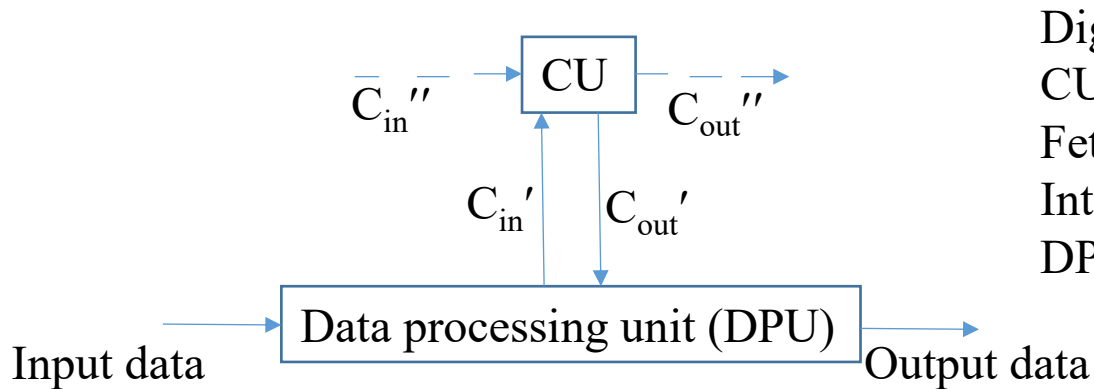
Control unit

Instruction sequencing

Methods by which instructions are selected for execution or how control of processor transfers from one instruction to other

Instruction interpretation

How a control unit interprets an instruction to determine the control signals to be issued?



Digital system is separated into two parts: CU and DPU

CU: Issue control signal or instruction to DPU

Fetch instruction from memory

Interpret instruction to determine the control signals to be sent to DPU

DPU: Performs certain operations on data

Control signal: Selects the function to be performed at specific time

Route the data through the appropriate functional units

C_{out}' : Directly control the operation of DPU

C_{in}' : Enable the data being processed to influence CU

Allowing data dependent decisions to be made

Indicate the occurrence of unusual condition such as errors like overflow in DPU

C_{out}'' : Transmitted to other CUs, may indicate status conditions such as “busy” or “operation completed”

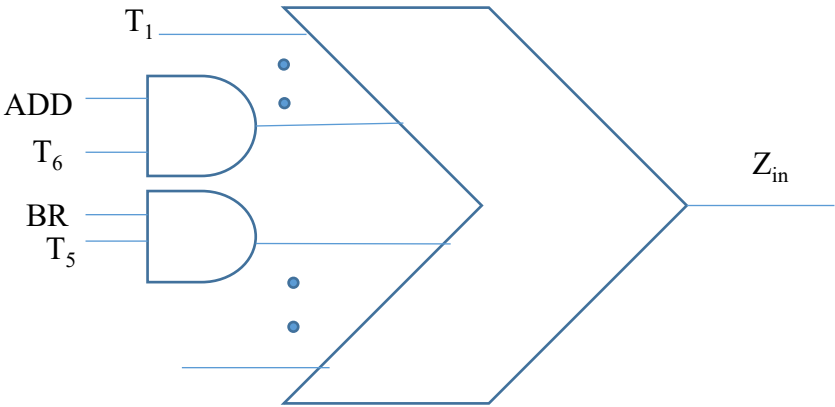
C_{in}'' : Receives from other CUs e.g. from a supervisor or controller, typically include start signal, stop signal, timing information

Example 1: Let control signal Z_{in} is required to be activated at T_1 time instant for all instructions, during T_6 for ADD instruction, during T_5 for conditional branch instruction etc.

$$Z_{in} = T_1 + (ADD)T_6 + (BR) T_5 + \dots$$

Example 2: Suppose C_5 control signal is required in the fetch cycle of all the instructions at time instant T_2 , during the execution of instruction I_1 at time instant T_4 , during the execution of instruction I_2, I_3 at time instant T_6 . P is 0 for fetch and 1 for execute.

$$C_5 = P' T_2 + P(I_1 T_4 + I_2 T_6 + I_3 T_6)$$



Design methods of hardwired CU

State table method: Behaviour required of a CU , like that of any finite state sequential machine, can be represented by a state table, C_{in} and C_{out} determine the input and output variables of CU

States	I_1	I_2				I_m
S_1	$S_{11}Z_{11}$	$S_{12}Z_{12}$				$S_{1m}Z_{1m}$
S_2	$S_{21}Z_{21}$	$S_{22}Z_{22}$				$S_{2m}Z_{2m}$
•	•	•				
•	•	•				
•	•	•				
S_n	$S_{n1}Z_{n1}$	$S_{n2}Z_{n2}$				$S_{nm}Z_{nm}$

$\{S_i\} \rightarrow$ Set of internal states of the machine as defined by the rows of the state table

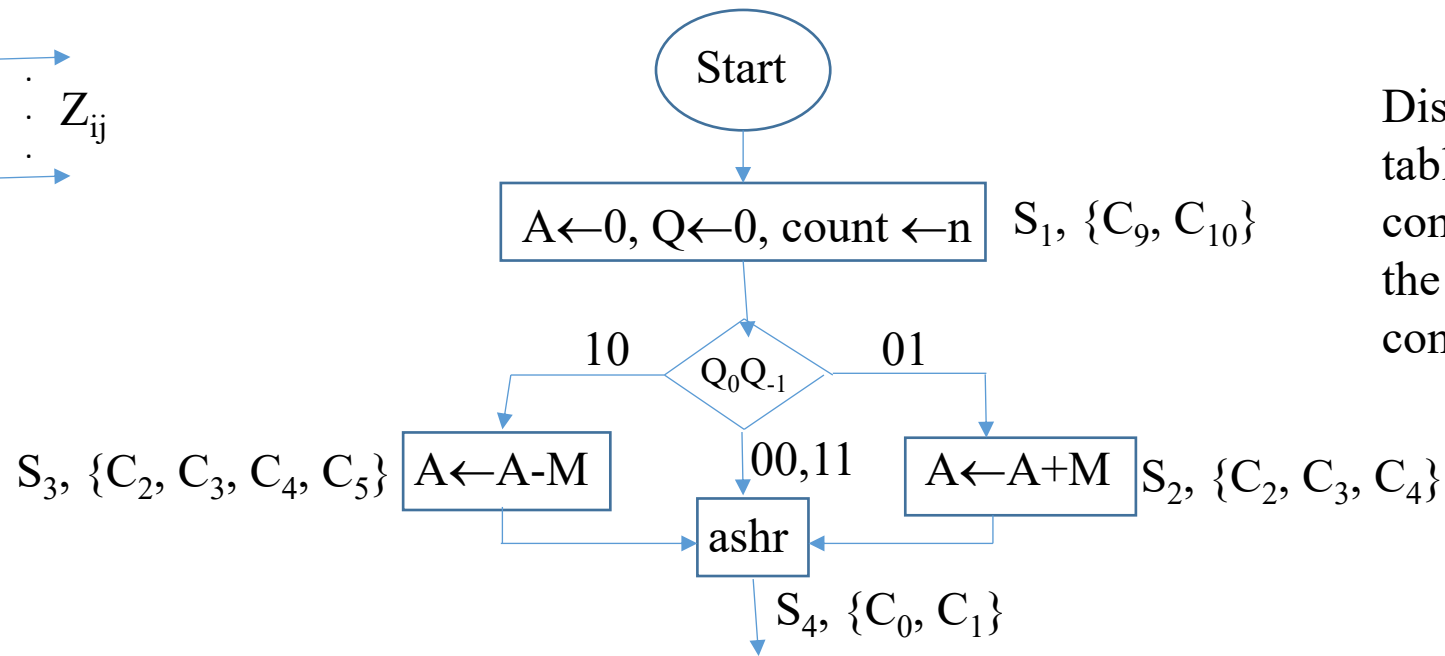
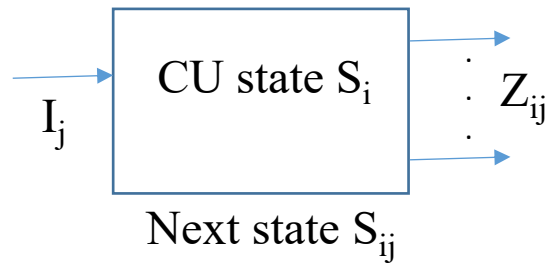
Internal state $\rightarrow$ Determined by the information stored in the unit at discrete points of time (clock time)

Column corresponds to the set of external signal to CU i.e. C_{in}

$S_{ij}Z_{ij} \rightarrow$ Entry in row S_i and column I_j

Z_{ij} : Denotes the set of output signal from C_{out} that are activated due to the application of I_j to control unit when it is in state S_i

S_{ij} : Denotes the next state of CU



Disadvantage: Size of state table and amount of computation increase with the number of state and input combinations

2. Delay element method: Uses clocked delay element for control signal timing

Consider the problem of generating the following sequence of control signals at time $t_1, t_2, \dots, t_n$ using hardwired CU

t_1 : Activate $\{C_{1j}\}$;

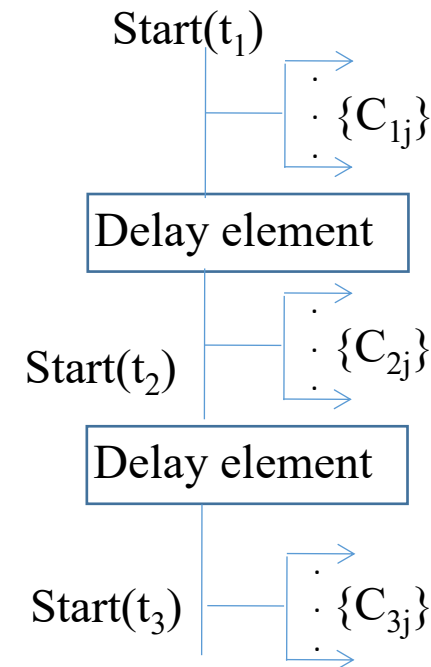
t_2 : Activate $\{C_{2j}\}$;

.

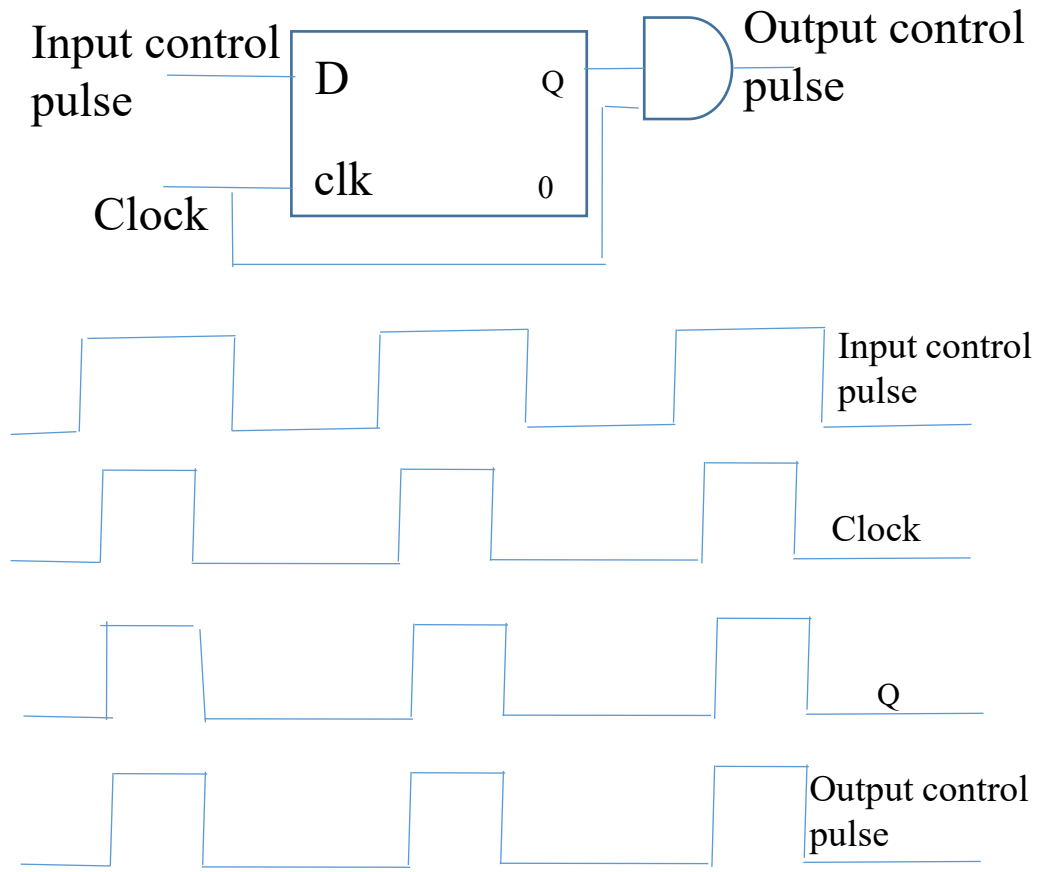
.

.

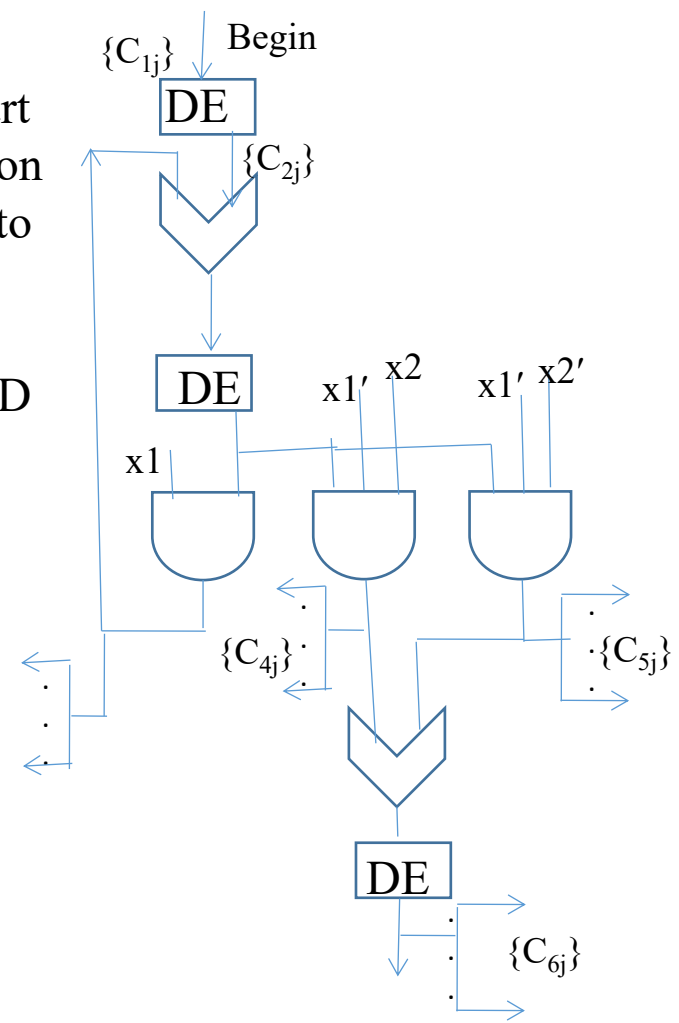
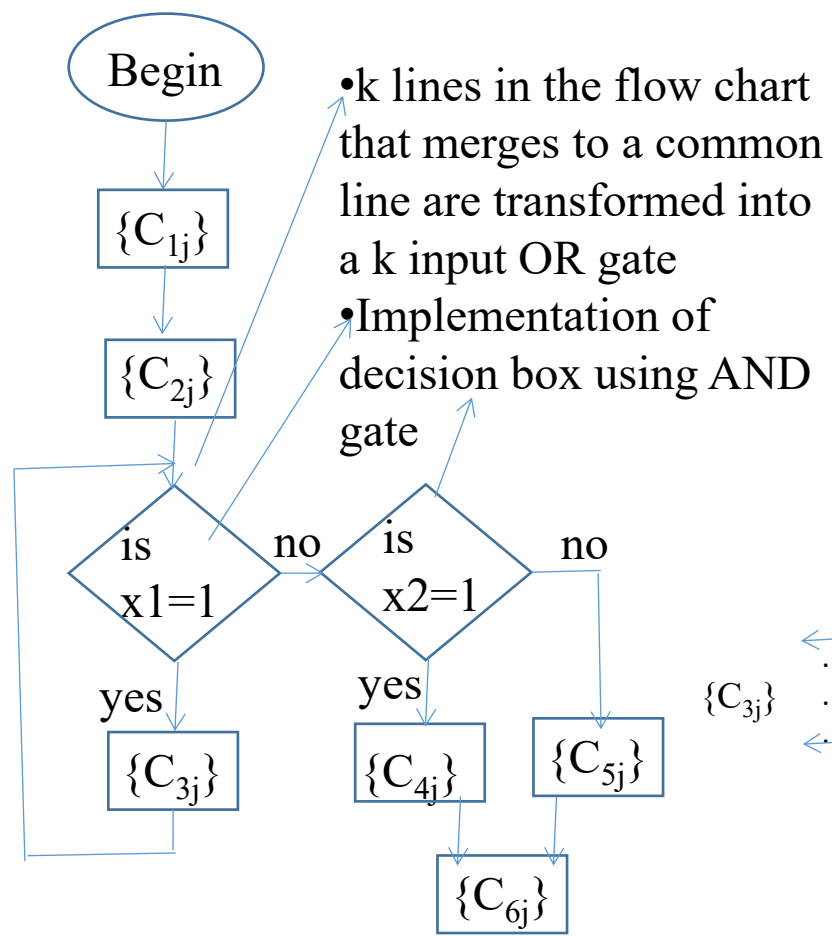
t_n : Activate $\{C_{nj}\}$;



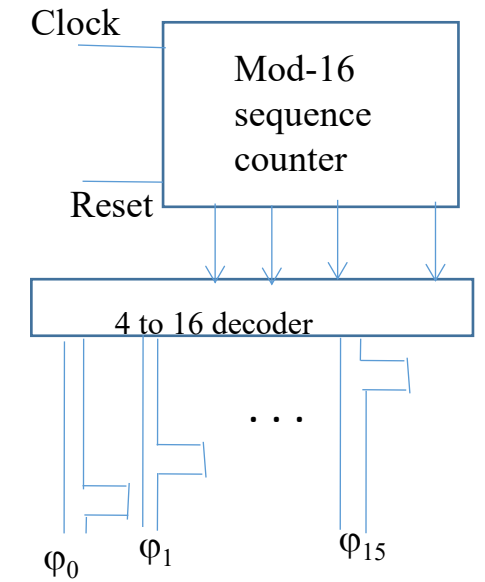
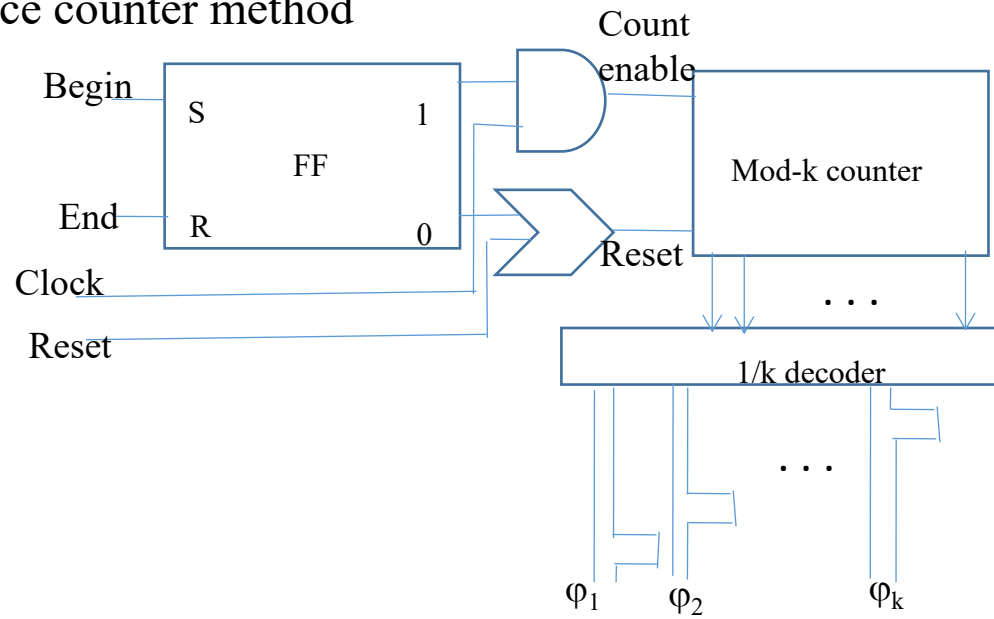
Synchronous delay element for CU design



Flow chart showing the control signals to be activated



3. Sequence counter method

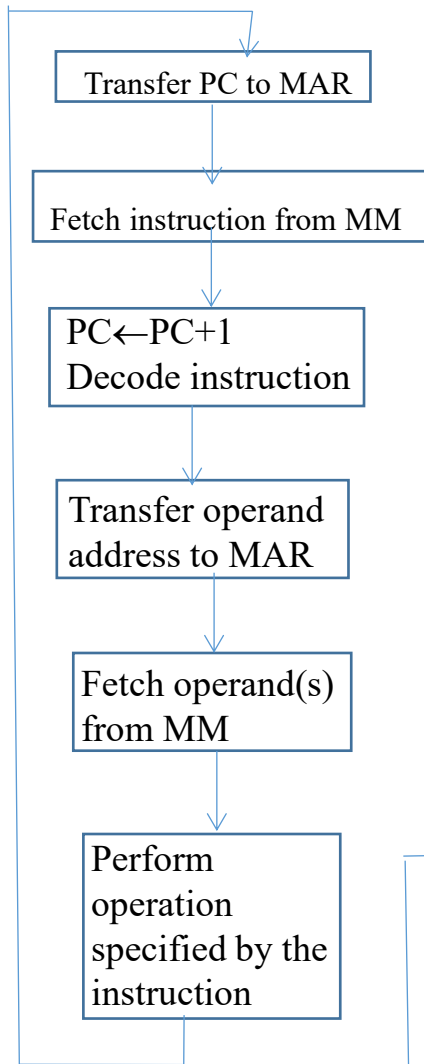


Begin=1, End=0, FF output = 1, Count enable = clock, counter goes on counting k clock pulses, for each count value one of the decoder output is high

Counter count value = i at i^{th} clock pulse, ϕ_i phase signal at the decoder output is activated, ϕ_i activates a set of control signals in step i of every instruction cycle

A pulse at the end line disconnects clock and resets the counter

CPU behaviour represented as a single closed loop



Each pass through the loop constitutes an instruction cycle

Assuming that each step can be performed in an appropriately chosen clock period

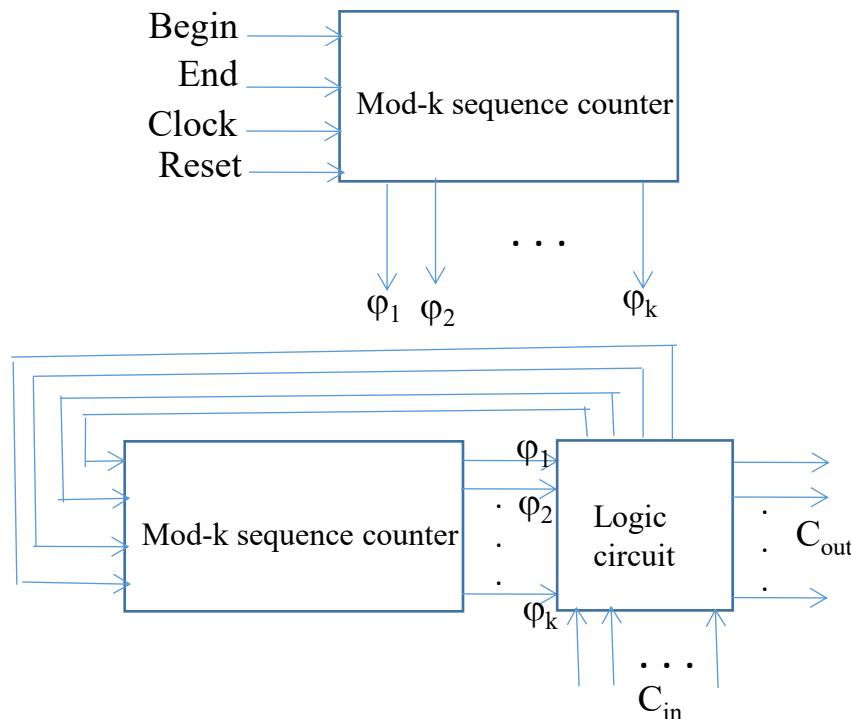
Modulo-6 counter may be used to build this

Each signal ϕ_i activates some set of control lines in step i of every instruction cycle

Operations performed in step i can be varied depending upon the certain control signals and condition variables applied to CU through C_{in}' , C_{in}''

$$C_{in} = \{C_{in}', C_{in}''\}$$

A logic circuit is needed to combine C_{in} with the timing signal $\{\phi_i\}$ generated by the sequence counter



At i^{th} clock pulse ϕ_i output of sequence counter is activated to generate the desired control signals at step i of every instruction cycle

If it is required to vary the operations performed in step i depending on certain control signals or condition variables applied to the CU ($C_{in} = \{C_{in}', C_{in}''\}$), a logic circuit combines C_{in} with the timing signal $\{\phi_i\}$ generated by the sequence counter.

Disadvantage of hardwired CU

Hardwired CU becomes complicated in case of large number of instructions and control lines, costly, difficult to design
Inherently inflexible if it is required to correct design errors or modify instruction set, needs redesign of CU

Microprogrammed CU

Instruction/Operation: Instruction is executed to do operation. For example, ADD R1,X is an instruction. It does add operation. It adds the content of register R1 and the content of the memory location X.

Microinstruction/Micro operation:

Add operation in the instruction ADD R1, X is divided into a few micro operation as follows

- $MAR \leftarrow PC$
- $MDR \leftarrow (M(MAR)), PC \leftarrow PC+1$
- $IR \leftarrow MDR$
- $Control\ block \leftarrow IR(opcode)$
- $MAR \leftarrow IR(address) \text{ ----} address = X$
- $MDR \leftarrow operand = data\ at\ X$
- $R1 \leftarrow R1 + (MDR)$

To execute each micro operation a set of control signal is required to activate

This set of control signal is activated by microinstruction

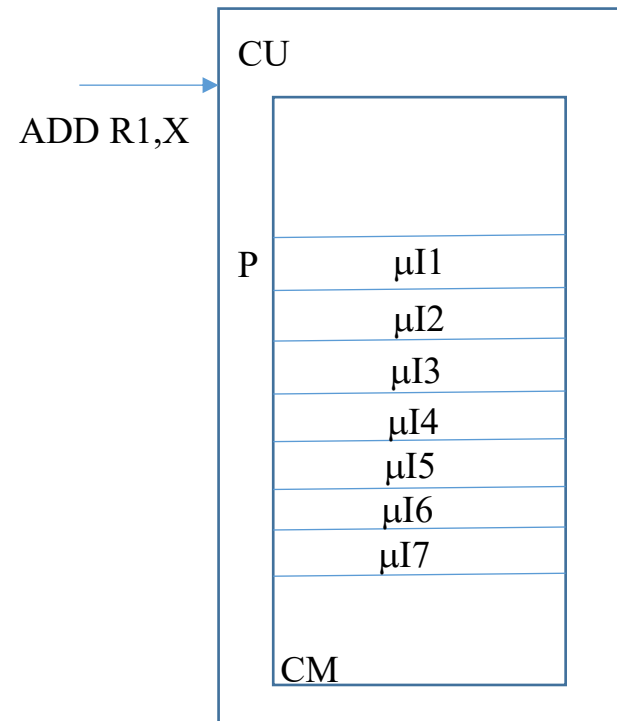
Hence to execute each micro operation, one microinstruction is required. For example, add operation in ADD R1, X is divided into 7 micro operations, and so 7 microinstructions are required to execute 7 micro operations.

Microprogram, microinstruction, control memory (CM)

Microprogram = a sequence of microinstructions

ADD R1, X $\rightarrow$ instruction that does addition operation, divided into 7 micro operations which needs 7 sets of control signal for execution and hence 7 microinstructions.

So the microprogram corresponding to the instruction ADD R1, X contains 7 microinstructions. This microprogram resides in CM with starting address say P.

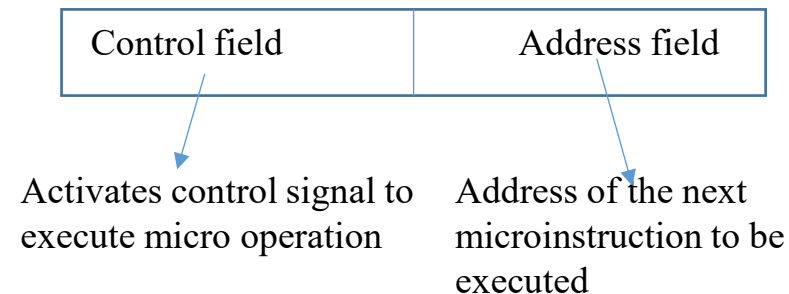


CU receives ADD R1, X;

CU computes the starting address of the corresponding microprogram in CM i.e. P from the opcode of this instruction, executes all the microinstructions in sequence from address P.

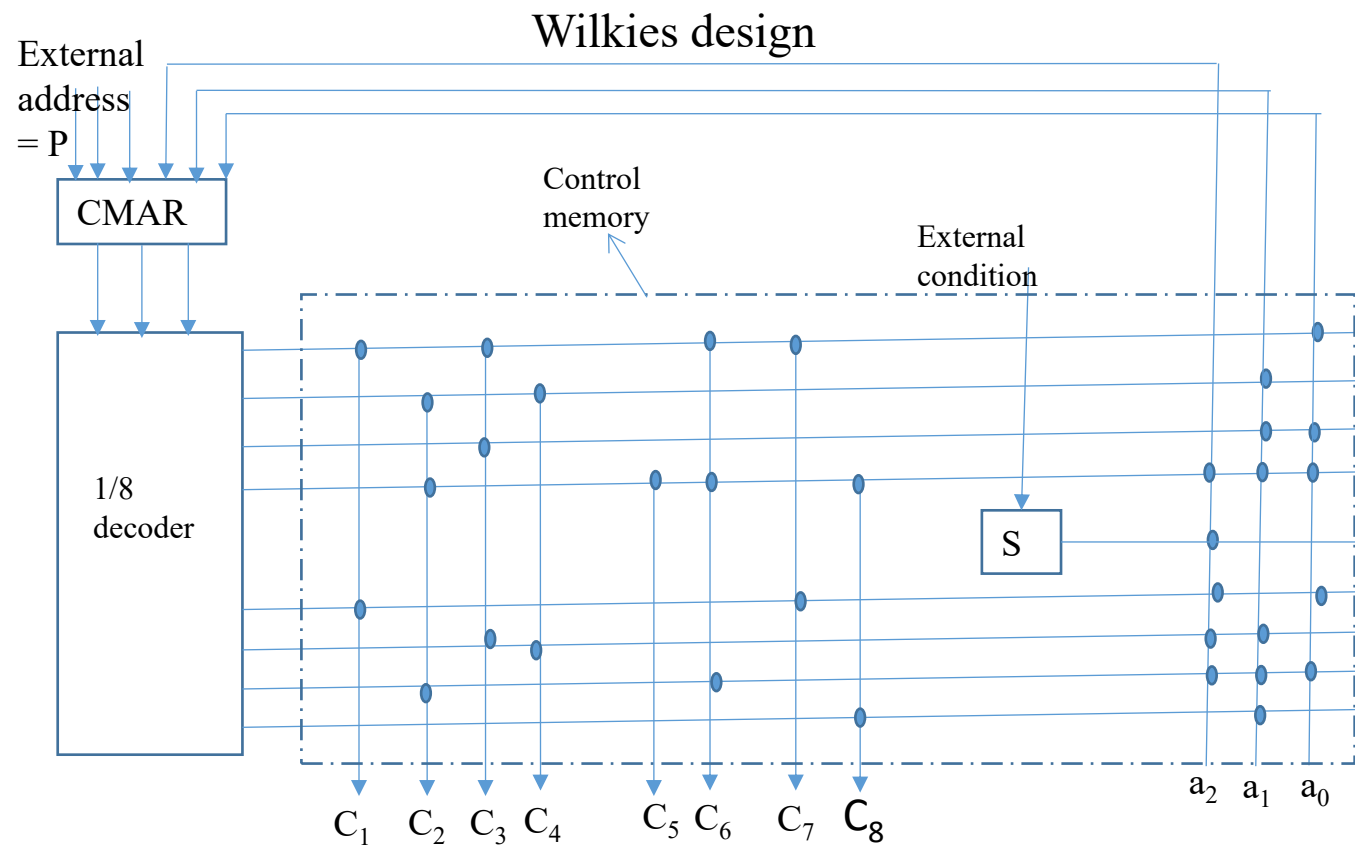
The execution of 7 microinstructions activates 7 sets of control signals to execute 7 micro operations

Format of microinstruction



CM: ROM if the instruction set of a processor is permanent

WCM (Writeable CM) allows to change the content of CM dynamically



Each row indicate one microinstruction

Top most horizontal line represents the microinstruction at address 000

External address source stores 000 in CMAR

Microinstruction at address 000 in CM is selected , control lines C₁, C₃, C₆, C₇ are activated

Address field of the microinstruction at location 000 in CM contains 001 which is the address of the next microinstruction

External condition: Any CU must be able to give response to external signals or conditions

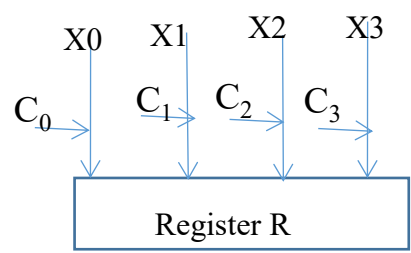
Switch S is controlled by an external condition FF

Address field of 4th microinstruction contains 111 whereas the address field corresponding to switch S contains 100. So depending upon the external condition either microinstruction at location 111 or 100 in CM is executed.

	Control field	Address field
μI1	C ₁ , C ₃ , C ₆ , C ₇	001
μI2	C ₂ , C ₄	010
μI3	C ₃	011
μI4	C ₂ , C ₅ , C ₆ , C ₈	111
μI5	C ₁ , C ₇	101
μI6	C ₃ , C ₄	110
μI7	C ₂ , C ₆	111
μI8	C ₈	010

Example:

Horizontal microinstruction



← Control field →				
	C ₀	C ₁	C ₂	C ₃
1	0	0	0	R ← X0
0	1	0	0	R ← X1
0	0	1	0	R ← X2
0	0	0	1	R ← X3
0	0	0	0	NOP

Operation LOAD R i.e. load the register R. Suppose the register R can be loaded from four different sources

R ← X0, control signal C₀ is activated

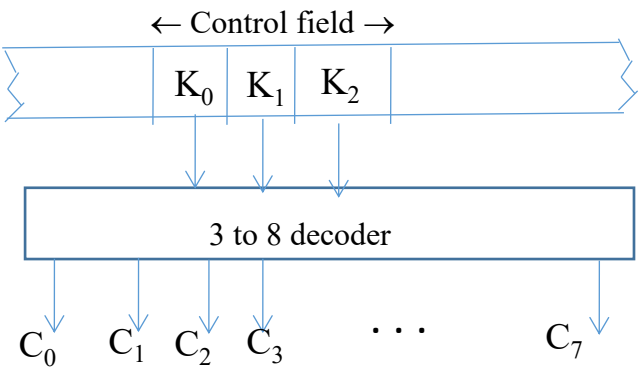
R ← X1, control signal C₁ is activated

R ← X2, control signal C₂ is activated

R ← X3, control signal C₃ is activated

Five micro operations

Vertical microinstruction



C₀ = 1 when K₀K₁K₂ = 001

C₁ = 1 when K₀K₁K₂ = 010

C₂ = 1 when K₀K₁K₂ = 011

C₃ = 1 when K₀K₁K₂ = 100

- Advantage of vertical microinstruction format over horizontal microinstruction format
- Reduces length of control field by 1 bit at the cost of an extra decoder
 - Short format
 - Needs encoding
 - N independent control signals for N independent micro operations can be encoded in a control field of log₂(N+1) bits

Fig. Horizontal and vertical microinstruction format

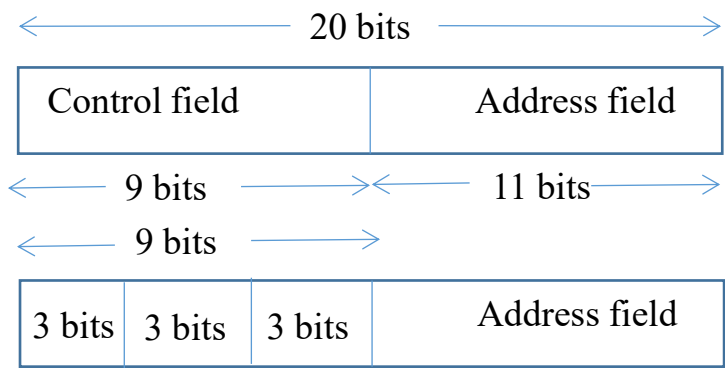
Parallelism in microinstruction

One instruction executes one operation. Let this operation is divided into N micro operations. So N number of microinstructions are required to generate N set of control signals. As with the number of micro operation number of microinstruction increases, size of the microprogram also increases which needs more storage space in CM.

Parallelism in microinstructions is used to reduce the number of microinstructions and hence to reduce the size of CM.

Definition: In microprogramming level independent micro operations are available and executed in parallel. Hence one microinstruction can specify control signals that are required to execute multiple independent micro operations.

Example of dependent micro operation: $AC \leftarrow 0, AC \leftarrow AC-DR$
Example of independent micro operation: $MDR \leftarrow (M(MAR)), PC \leftarrow PC+1$



CM: Number of memory locations = 2^{11}
Each memory location contains 20 bits microinstruction
Size of CM = $2^{11} \times 20$ bits
9 bits control field is divided in 3 field that is 3 sets of control signals for 3 independent micro operations can be activated by a single microinstruction.
Degree of parallelism = 3

F1=F2=F3=000 signifies NOP		
F1 = 001 → AC ← AC+DR	F2 = 001 → AC ← AC-DR	F3 = 001 → AC ←
F1 = 010 → AC ← 0	F2 = 010 → AC OR DR	AC⊕DR
F1 = 011 → AC ← AC+1	.	F3 = 010 → AC← AC'
.	.	F3 = 011 → PC ← MAR
.	.	.
.	.	.

Let us consider two micro operations as $PC \leftarrow MAR$ and $AC \leftarrow AC+DR$. These two micro operations are independent. So the control signals that are required to activate for the execution of the two micro operations can be specified in a single microinstruction.

← Control field →

001	000	011	Address field
-----	-----	-----	---------------

Example:
 Total number of micro operation = 46
 Degree of parallelism = 2
 What is the size of two subfields in 9 bits control field of microinstruction?
 $2^4 + 2^5 = 16 + 32 = 48 = 46+2$
 ↓
 NOP

Control memory optimization

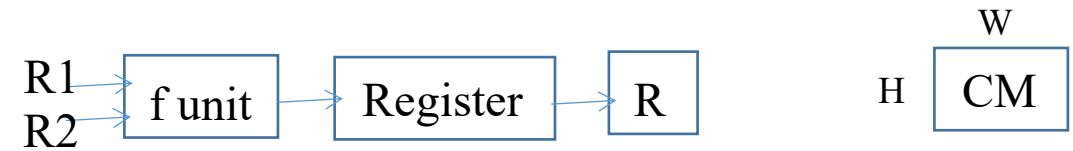
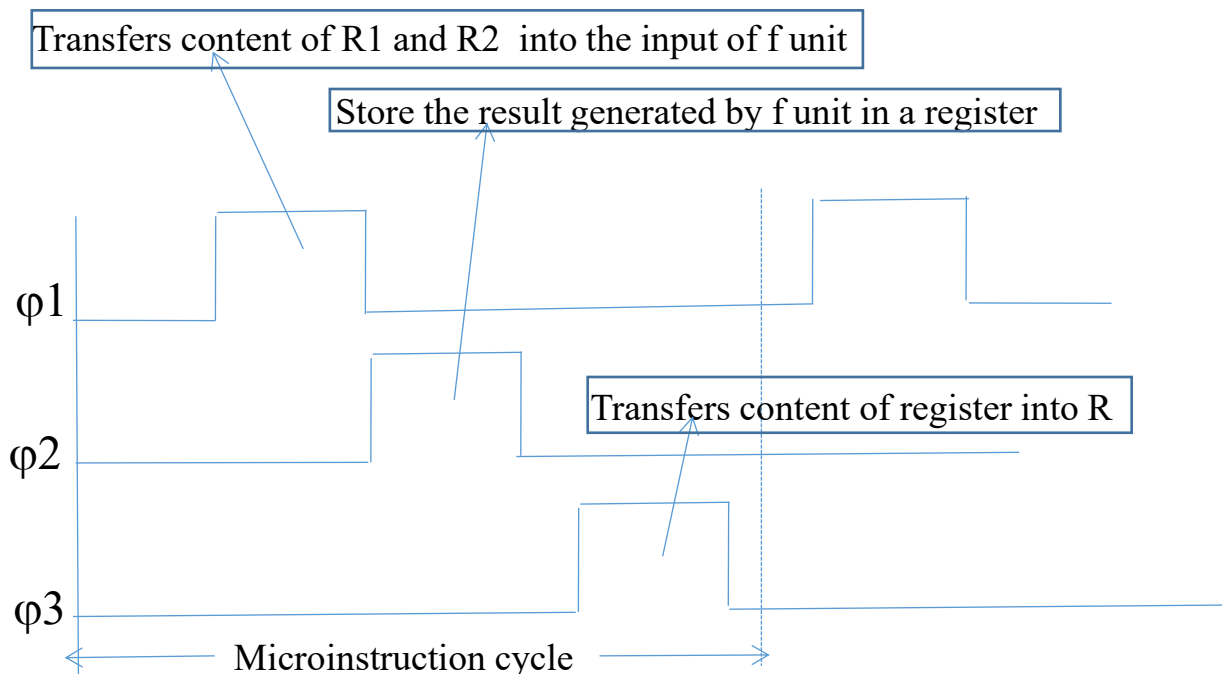
Monophase: So far we have assumed that each microinstruction generates a set of control signals which are active for the duration of the microinstruction execution cycle

Clock period = microinstruction cycle period

Polyphase: The number of microinstructions needed to specify a particular operation can often be reduced by dividing the microinstruction cycle into several sequential phases. Each control signal is typically remains active during only one of the phases. This phase permits a single microinstruction to specify a sequence of micro operations.

Example: Micro operation , $R \leftarrow f(R1, R2)$

Transfers content of R1 and R2 into the input of f unit



Microcode compaction

Reduce H by merging two or more micro operations into a single microinstruction

Basic principle of merging

m1 : R2 ← F(R1)

m2 : R3 ← F(R2)

m3 : R4 ← F(R3)

m1 and m2 have direct data dependency

m1 and m3 have indirect data dependency

m2 and m3 have direct data dependency

m1, m2, m3 can't be executed in parallel, can never be placed in the same monophase microinstruction, may appear in different phases of the same polyphase microinstruction

Data dependent micro operations can be executed sequentially in the proper time sequence within a single microinstruction cycle

Such data dependent micro operations are called weakly data dependent

Example: Microinstruction cycle is divided into 3 subphases				
Microinstruction before compaction				Microinstruction after compaction
Name	Micro operation	Phase used	Resource used	
m1	RLEFT ← AC	φ1	Adder Shifter Bus	I1 = {m1}
m2	RRIGHT ← 1	φ1		I1 = {m1, m2}
m3	ROUT ← RLEFT+RRIGHT	φ2		I1 = {m1, m2, m3}
m4	MAR←RIGHTSHIFT(ROUT)	φ3		I1 = {m1, m2, m3, m4}
m5	M(MAR) ← MDR	φ2	Bus	I2={m5}
m6	RLEFT←1	φ1		I2={m5, m6}
m7	RRIGHT ← R0	φ1		I2={m5, m6, m7}

H = 2

Microinstruction after compaction

m1, m2 are independent and so can be executed in the same phase ϕ_1 and can be included in the same microinstruction I1
m1, m2, m3 are dependent but m3 needs different phase ϕ_2 , so m3 can be included in I1 (due to weak data dependency)
m3 and m4 are dependent but they need different phase and resource, hence m4 can be included in I1
m4 and m5 are strongly data dependent, m5 needs ϕ_2 for execution, so need to be placed in different microinstruction I2
m1 and m6, m2 and m7 are strongly data dependent. So m6 and m7 can't be placed in I1. But m6 and m7 are independent on m5 and so can be placed in I2

Example: Microinstruction cycle is divided into 3 sbphases

Microinstruction before compaction				Microinstruction after compaction
Name	Micro operation	Phase used	Resource used	
m1	$A \leftarrow R1$	ϕ_1	Adder	I1 = {m1}
m2	$C \leftarrow A+R2$	ϕ_2		I2 = {m2}
m3	$R3 \leftarrow D$	ϕ_2	Adder	I1 = {m1, m3}
m4	$R6 \leftarrow R6+1$	ϕ_2		I1 = {m1, m3, m4}
m5	$R7 \leftarrow R3 \text{ OR } R6$	ϕ_2	ALU	I2={m2, m5}

m1 and m2 are strongly data dependent
m2 and m4 are resource dependent, both needs ϕ_2
m4 and m5 are data dependent, both needs ϕ_2

As m4 needs adder like m2

$H = 2$

Minimizing W

Objective is to derive a format of the microinstruction control field such that the total number of bits in the control field is a minimum i.e. we want a control field encoding method that uses as few bits as possible

Steps of operation

- Determine the set of maximal compatibility class (MCC)
- Determine all minimal set of MCCs that include each control signal. Each of these sets is called minimal MCC cover
- Inspect each minimal MCC cover $\{C_i\}$ in turn and determine all way of including each control signal in exactly one subset of some C_i .

Compatibility class

- C_i = control signal, I_j = microinstruction
- $C_i \in I_j \rightarrow C_i$ is activated by I_j
- Control signals C_1 and C_2 are compatible if $C_1 \in I_j$ but $C_2 \in I_j$ and vice versa

Maximal compatibility class : Defined as compatibility classes to which no control signal can be added without introducing a pair of incompatible control signals

- Step-1: Determine the set of MCCs
- First determine all compatibility classes containing two control signals
- Second determine all compatibility classes containing three control signals
- Process terminates when no new compatibility classes can be formed
- S1: a b c d e f g h
- S2: bd be bh cd de dg dh ef eg fg fh gh
- S3: bde bdh deg dgh efg fgh
- Rule for forming S_{i+1} from S_i :
 - Add a new control signal to a, say {a,b}
 - Pair {a,b} is not compatible so check for {a,c} and so on

Example	
Microinstruction	Control signals
I1	a, b, c, g
I2	a, c, e, h
I3	a, d, f
I4	b, c, f

Step-2: Determination of all minimal set of MCCs that include each control signal. Each of these sets is called minimal MCC cover

Uses cover table that contains a row for each MCC (C_i) and column for each control signal (C_j)

X is placed at the intersection of row C_i and column C_j if $C_j \in C_i$ i.e. C_i covers C_j .

Control Signal C_j MCC C_i	a	b	c	d	e	f	g	h
$C1 = a$	X							
$C2 = cd$			X	X				
$C3 = bde$		X		X	X			
$C4 = bdh$		X		X				X
$C5 = deg$				X	X		X	
$C6 = dgh$				X			X	X
$C7 = efg$					X	X	X	
$C8 = fgh$						X	X	X

Certain rows and columns can be deleted from the cover table to simplify determination of the minimal MCC covers

1. Column C_j contains only one X, which occurs in row C_i , C_i is said to be an essential MCC and must appear in every minimal MCC cover, since it only covers C_j

Example: Column a and column c contain only one X. So $C1$ and $C2$ are two essential MCC and must appear in every minimal MCC cover since $C1$ is only MCC that covers the control signal a and $C2$ is only MCC that covers the control signal c

2. All rows corresponding to essential MCCs can be deleted from the cover table and all columns with X in essential rows can be deleted

Hence deletes row $C1$, $C2$ and columns a, c, d.

3. If the cover table contains two identical columns then all but one of these columns can be deleted.

Example: there is no identical columns

4. Column C_i is said to dominate column C_j if C_i contains an X in every row where C_j contains an X and C_i contains more X than C_j

The dominating column can be deleted as every MCC that covers C_j automatically covers C_i

Example: d dominates b and c while g dominates f , hence deletes column g

5. Row C_i is said to dominate row C_j if C_i contains an X in every columns where C_j contains an X and C_i contains more X than C_j

In this case the dominating row C_j can be deleted, since C_i covers all the control signals that are covered by C_j

Example: There is no such dominating row.

Reduced cover table

Control Signal C_j MCC C_i	b	e	f	h
C3=bde	X	X		
C4=bdh	X			X
C5=deg		X		
C6=dgh				X
C7=efg		X	X	
C8=fgh			X	X

Step-3: Inspect each minimal MCC cover $\{C_i\}$ in turn and determine all ways of including each control signal in exactly one subset of some C_i

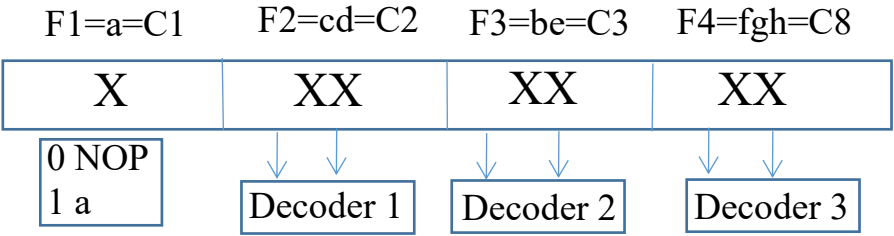
The minimal cover for this table contains two MCCs, possible choices are $\{C3, C8\}$ and $\{C4, C7\}$

Combining these with essential MCCs C1 and C2 the desired MCC solutions are $\{C1, C2, C3, C8\}$ and $\{C1, C2, C4, C7\}$.

Consider one of the two minimal MCC, say $\{C1, C2, C3, C8\}$.

There is only one way to cover a (by C1), b (by C3), c (by C2), e (by C3), f,g,h (by C8).

There is two way to cover d (by C2 and C3).



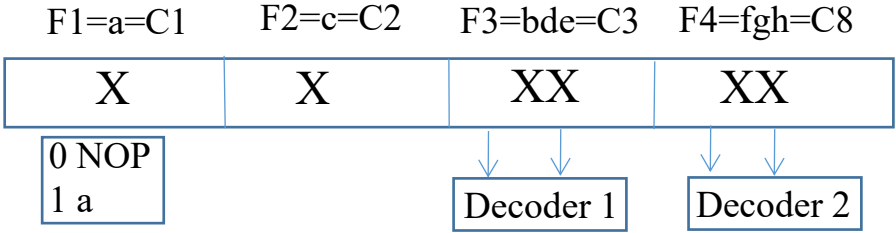
Decoder 1	Decoder 2	Decoder 3
00 → NOP	00 → NOP	00 → NOP
01 → d	01 → b	01 → f
10 → c	10 → e	10 → g
		11 → h

If we decide to cover d by C2 then delete d from C3 to obtain the partition {a, cd, be, fgh}.

F1, F2, F3, F4 are the four subfields in microinstruction control

W = width of the control field = 7 bits

Totally horizontal microinstruction format needs 8 bits in control field for 8 control signals without the use of any decoder. So vertical format needs 6 bits and horizontal format needs 8 bits in the control field.



Decoder 1	Decoder 2
00 → NOP	00 → NOP
01 → b	01 → f
10 → d	10 → g
11 → e	11 → h

If we want to cover d by C3 then delete d from C2 to obtain the partition {a, c, bde, fgh}

W = 6 bits, number of decoder reduces